

ANSWERS

ANSWER-A

Intelligent Agent – Warehouse Robot

A warehouse robot is an intelligent agent that moves around a warehouse, identifies products, picks and places items, and delivers them to the required location.

PEAS Description

PEAS	Description
P – Performance Measure	Correctly picking and delivering products, minimum time, minimum errors, avoiding collisions, and saving energy.
E – Environment	Warehouse, shelves, products, workers, other robots, obstacles, pathways, loading/unloading areas.
A – Actuators	Wheels/motors for movement, robotic arm for picking and placing products, gripper, and lifting mechanism.
S – Sensors	Camera, distance/proximity sensors, barcode/RFID scanner, GPS/position sensors, and obstacle sensors.

Formal Representation

Agent: Warehouse Robot

Task: Pick products → Navigate → Deliver products → Return/Continue next task.

PEAS = (Performance, Environment, Actuators, Sensors)

ANSWER-B

Type of Intelligent Agent for Warehouse Robot

The Utility-Based Agent is best suited for a warehouse robot.

A warehouse robot has to perform many tasks such as finding products, moving through the warehouse, avoiding obstacles, picking products, and delivering them to the correct location. A utility-based agent can compare different possible actions and choose the one that gives the best overall result.

Why Utility-Based Agent?

The robot can consider:

- Shortest route to the destination
- Safety from obstacles and workers
- Time required to complete the task
- Battery consumption
- Delivery accuracy

For example, if there are two routes, the robot can select a slightly longer route if it is much safer. Therefore, a utility-based agent is more suitable than a simple reflex or goal-based agent.

Simple Python Program

```
class WarehouseRobot:

    def choose_route(self, routes):
        best_route = ""
        highest_utility = -999

        for route, distance, safety in routes:
            utility = (safety * 10) - distance

            print(route, "Utility =", utility)

            if utility > highest_utility:
                highest_utility = utility
                best_route = route

        return best_route

robot = WarehouseRobot()

routes = [
    ("Route A", 10, 8),
    ("Route B", 6, 7),
    ("Route C", 8, 9)
]

best = robot.choose_route(routes)

print("\nBest Route:", best)
```

OUTPUT:

```
Route A Utility = 70
Route B Utility = 64
Route C Utility = 82

Best Route: Route C
```

ANSWER-3

Formal Search Problem – Warehouse Robot

The warehouse robot can be modeled as a **search problem** where the robot must find a path from its starting position to the location where a product must be delivered.

1. Initial State

The robot starts at a particular location in the warehouse with the required product.

Example:

Initial State = Robot at (0,0), Product picked

2. Goal State

The goal is to reach the required delivery location with the product.

Example:

Goal State = Robot at (4,4), Product delivered

3. Actions

The robot can perform the following actions:

- Move Up
- Move Down
- Move Left
- Move Right

4. Transition Model

The transition model describes how the warehouse robot's position changes after an action.

For example:

- Move Up $\rightarrow (x, y+1)$
- Move Down $\rightarrow (x, y-1)$
- Move Left $\rightarrow (x-1, y)$
- Move Right $\rightarrow (x+1, y)$

The robot cannot move outside the warehouse or through obstacles.

5. Path Cost Function

The path cost is the total cost of all movements.

For simplicity:

Cost of each movement = 1

Therefore:

Path Cost = Number of movements

The robot should find a path with the **minimum cost**.

Simple Python Program

```
from collections import deque

def warehouse_search(start, goal, warehouse):

    rows = len(warehouse)
    cols = len(warehouse[0])

    queue = deque([(start, [start])])
    visited = {start}

    actions = [
        (-1, 0), # Up
        (1, 0), # Down
        (0, -1), # Left
        (0, 1) # Right
    ]

    while queue:
        position, path = queue.popleft()

        if position == goal:
            return path

        x, y = position

        for dx, dy in actions:
            nx, ny = x + dx, y + dy

            if (0 <= nx < rows and
                0 <= ny < cols and
```

```

        0 <= ny < cols and
        warehouse[nx][ny] == 0 and
        (nx, ny) not in visited):

        visited.add((nx, ny))
        queue.append(((nx, ny), path + [(nx, ny)]))

    return None

# 0 = free space, 1 = obstacle
warehouse = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)
goal = (4, 4)

path = warehouse_search(start, goal, warehouse)

print("Initial State:", start)
print("Goal State:", goal)
print("Path:", path)
print("Path Cost:", len(path) - 1)

```

OUTPUT:

```

Initial State: (0, 0)
Goal State: (4, 4)
Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Path Cost: 8

```

ANSWER-D

Breadth-First Search (BFS)

BFS explores the nodes **level by level** using a FIFO queue.

Step-by-Step Expansion

Step	Expanded Node	Frontier / Queue
1	(1,1)	(1,2), (2,1)
2	(1,2)	(2,1), (1,3), (2,2)
3	(2,1)	(1,3), (2,2), (3,1)
4	(1,3)	(2,2), (3,1), (1,4), (2,3)
5	(2,2)	(3,1), (1,4), (2,3), (3,2)
6	(3,1)	(1,4), (2,3), (3,2), (4,1)
7	(1,4)	(2,3), (3,2), (4,1), (1,5), (2,4)
8	(2,3)	(3,2), (4,1), (1,5), (2,4), (3,3)
9	(3,2)	(4,1), (1,5), (2,4), (3,3), (4,2)
10	(4,1)	(1,5), (2,4), (3,3), (4,2), (5,1)
11	(1,5)	(2,4), (3,3), (4,2), (5,1), (2,5)
12	(2,4)	(3,3), (4,2), (5,1), (2,5), (3,4)
13	(3,3)	(4,2), (5,1), (2,5), (3,4), (4,3)
14	(4,2)	(5,1), (2,5), (3,4), (4,3), (5,2)
15	(5,1)	(2,5), (3,4), (4,3), (5,2)
16	(2,5)	(3,4), (4,3), (5,2), (3,5)
17	(3,4)	(4,3), (5,2), (3,5), (4,4)
18	(4,3)	(5,2), (3,5), (4,4), (5,3)
19	(5,2)	(3,5), (4,4), (5,3)
20	(3,5)	(4,4), (5,3), (4,5)

Step	Expanded Node	Frontier / Queue
21	(4,4)	(5,3), (4,5), (5,4)
22	(5,3)	(4,5), (5,4)
23	(4,5)	(5,4), (5,5)
24	(5,4)	(5,5)
25	(5,5) Goal	Empty

Final Path

(1,1) → (1,2) → (1,3) → (1,4) → (1,5) → (2,5) → (3,5) → (4,5) → (5,5)

Total Cost

Each movement has a cost of **1**.

Total Cost = 8

Result

BFS successfully finds the **shortest path** from **(1,1) to (5,5)** with a total path cost of **8**.

SIMPLE PROGRAM:

```
from collections import deque

def bfs(start, goal):
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        current, path = queue.popleft()

        if current == goal:
            return path

        x, y = current

        for dx, dy in [(0, 1), (1, 0), (0, -1), (-1, 0)]:
            new = (x + dx, y + dy)

            if 1 <= new[0] <= 5 and 1 <= new[1] <= 5:
                if new not in visited:
                    visited.add(new)
                    queue.append((new, path + [new]))

start = (1, 1)
goal = (5, 5)

path = bfs(start, goal)

print("Path:", path)
print("Total Cost:", len(path) - 1)
```

OUTPUT:

```
Path: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5)]
Total Cost: 8
```

ANSWER-E

Depth-First Search (DFS)

Initial State: (1,1)

Goal State: (5,5)

Movement: Up, Down, Left, Right

Cost of each movement: 1

Using DFS with the order **Right** → **Down** → **Left** → **Up**, the robot explores one path deeply before backtracking.

Step-by-Step Stack

Step	Expanded Node	Stack
1	(1,1)	(1,2), (2,1)
2	(1,2)	(1,3), (2,2), (2,1)
3	(1,3)	(1,4), (2,3), (2,2), (2,1)
4	(1,4)	(1,5), (2,4), (2,3), (2,2), (2,1)
5	(1,5)	(2,5), (2,4), (2,3), (2,2), (2,1)
6	(2,5)	(3,5), (2,4), (2,3), (2,2), (2,1)
7	(3,5)	(4,5), (3,4), (2,4), (2,3), (2,2), (2,1)
8	(4,5)	(5,5), (4,4), (3,4), (2,4), (2,3), (2,2), (2,1)
9	(5,5)	Goal reached

Final Path

(1,1) → (1,2) → (1,3) → (1,4) → (1,5) → (2,5) → (3,5) → (4,5) → (5,5)

Total Cost

Number of movements = 8

Total Path Cost = 8

Simple Python Program:

```
def dfs(start, goal):
    stack = [(start, [start])]
    visited = set()

    while stack:
        current, path = stack.pop()

        if current in visited:
            continue

        visited.add(current)
        print("Expanded:", current)

        if current == goal:
            return path

        x, y = current

        moves = [(0, 1), (1, 0), (0, -1), (-1, 0)]

        for dx, dy in reversed(moves):
            new = (x + dx, y + dy)

            if 1 <= new[0] <= 5 and 1 <= new[1] <= 5:
                stack.append((new, path + [new]))

    path = dfs((1, 1), (5, 5))

    print("Final Path:", path)
    print("Total Cost:", len(path) - 1)
```

OUTPUT

```
Expanded: (1, 1)
Expanded: (1, 2)
Expanded: (1, 3)
Expanded: (1, 4)
Expanded: (1, 5)
Expanded: (2, 5)
Expanded: (3, 5)
Expanded: (4, 5)
Expanded: (5, 5)
Final Path: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5)]
Total Cost: 8
```

Conclusion

The warehouse robot was successfully modeled as an **intelligent agent** using the PEAS framework. A **utility-based agent** was selected because it can choose the safest and most efficient actions. The warehouse task was then represented as a **search problem** using the initial state, goal state, actions, transition model, and path cost.

BFS found the shortest path from (1,1) to (5,5) with a cost of **8**, while **DFS** also reached the goal with a cost of **8** for the given movement order. Thus, search algorithms can help the warehouse robot navigate efficiently from its starting location to the required destination.
